

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS:

Total Marks: 30

1.A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

2.An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Code of Conduct:

I N. Meghana(192411129) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N. Meghana

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

INDUSTRY PROBLEM

1 – SOAP-BASED HOSPITAL MANAGEMENT WEB SERVICE

A hospital management system contains different departments such as reception, outpatient services, laboratory, pharmacy, billing, and medical records. These departments may use different applications, but they still need to exchange patient information correctly. A SOAP-based web service can provide a common communication mechanism between these applications. The service can handle patient registration, appointment scheduling, and retrieval or updating of medical records in a structured and interoperable manner.

1.1 Proposed System Design

The proposed system uses a client-server web service model. Hospital department applications act as clients and communicate with a central SOAP web service. The service receives a SOAP request, checks the request, performs the required operation on the hospital database, and sends a SOAP response back to the client. This arrangement avoids direct database access by every department and provides a common interface for exchanging information.

- Client applications: reception, doctor, laboratory, pharmacy, billing, and administration systems.
- SOAP web service: provides standard operations for patient and appointment management.
- Application/service layer: validates requests and applies hospital business rules.
- Database: stores patient details, appointments, medical records, and related information.
- Security layer: provides authentication, authorization, confidentiality, and message protection.

1.2 Main Operations

The web service can expose a set of operations through which departments can communicate with the hospital system. Important operations include:

- addPatient – Registers a new patient using patient ID, name, date of birth, contact information, and address.
- getPatient – Retrieves basic information about a patient using the patient ID.
- scheduleAppointment – Creates an appointment between a patient and a doctor for a selected date and time.
- getAppointments – Returns appointments associated with a patient or doctor.
- getMedicalRecords – Retrieves medical records for authorized users.
- updateMedicalRecord – Adds or updates information in a patient's medical record.

- cancelAppointment – Cancels an existing appointment after validating the appointment and user permissions.

1.3 Structure of a SOAP Message

A SOAP message is an XML document with a defined structure. The main parts are Envelope, Header, and Body. The Envelope is mandatory and identifies the document as a SOAP message. The Header is optional and carries additional information such as authentication, transaction details, or security information. The Body contains the actual request or response data.

A simplified SOAP request for adding a patient is:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:h="http://hospital.example.com/service">
  <soap:Header>
    <h:Authentication><h:UserId>reception01</h:UserId>
    <h:Token>secure-token</h:Token></h:Authentication>
  </soap:Header>
  <soap:Body>
    <h:addPatient>
      <h:patientId>P1001</h:patientId>
      <h:name>Arun Kumar</h:name>
      <h:dateOfBirth>2001-05-10</h:dateOfBirth>
      <h:phone>9876543210</h:phone>
    </h:addPatient>
  </soap:Body>
</soap:Envelope>
```

1.4 SOAP Response

After receiving the request, the server validates the message and stores the patient information if all required conditions are satisfied. It then returns a SOAP response indicating the result.

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:h="http://hospital.example.com/service">
  <soap:Body>
    <h:addPatientResponse>
      <h:status>SUCCESS</h:status>
      <h:message>Patient registered successfully</h:message>
      <h:patientId>P1001</h:patientId>
    </h:addPatientResponse>
  </soap:Body>
</soap:Envelope>
```

```
</h:addPatientResponse>  
</soap:Body>  
</soap:Envelope>
```

1.5 How Data Is Exchanged Between Client and Server

1. The hospital department selects an operation.
2. The client creates a SOAP XML request containing the required information.
3. The request is sent to the service endpoint using HTTP/HTTPS.
4. The service checks the envelope, namespace, authentication, and body.
5. The service executes the operation and communicates with the database.
6. The server creates a SOAP response or SOAP Fault.
7. The client receives and displays the result to the authorized user.

1.6 Error Handling Using SOAP Fault

If a request cannot be processed, the service can use a SOAP Fault to describe the error. Examples include an invalid patient ID, missing required field, unauthorized user, database failure, or incorrect SOAP structure. A fault response allows the client application to handle the problem in a predictable way.

1.7 Why SOAP Is Suitable for the Hospital System

SOAP is appropriate because hospital applications need structured communication, clear service contracts, and reliable handling of business operations. SOAP messages are XML-based and can be processed across different programming languages and platforms. SOAP also supports standards and extensions for security, transactions, and reliable messaging. HTTPS can protect the communication channel, while WS-Security can be used when message-level security is required.

SOAP also provides a clear error mechanism through SOAP Faults. This is useful in a hospital environment because an application must distinguish between a successful request, invalid input, authentication problems, and server-side failures. Therefore, SOAP provides a suitable foundation for secure and interoperable communication between hospital departments.

2 – WSDL STRUCTURE FOR ENTERPRISE WEB SERVICE

WSDL, or Web Services Description Language, is an XML-based description of a web service. It provides a formal contract that tells a client what operations are available, what input and output messages are expected, how messages are transmitted, and where the service can be

accessed. In an enterprise environment, this description allows different systems to integrate without needing to know the internal implementation of the service.

2.1 Proposed Web Service

Consider an enterprise data service called EmployeeDataService. It provides operations to retrieve employee information and update employee details. The WSDL defines the data types, messages, operations, communication binding, and service endpoint.

2.2 Key Components of WSDL

- Types – Defines the data types used by the service, normally through XML Schema.
- Message – Describes the data exchanged between client and service.
- portType – Defines the abstract interface and groups the operations.
- Binding – Specifies how the operations use a concrete protocol such as SOAP over HTTP.
- Service – Groups service endpoints and provides the address at which the service can be accessed.

2.3 Example WSDL Structure

```
<definitions name="EmployeeDataService"
  targetNamespace="http://example.com/employee"
  xmlns:tns="http://example.com/employee"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns="http://schemas.xmlsoap.org/wsdl/">

  <types>
    <xsd:schema targetNamespace="http://example.com/employee">
      <xsd:element name="employeeId" type="xsd:string"/>
      <xsd:element name="employeeName" type="xsd:string"/>
    </xsd:schema>
  </types>

  <message name="GetEmployeeRequest">
    <part name="employeeId" element="tns:employeeId"/>
  </message>

  <message name="GetEmployeeResponse">
    <part name="employeeName" element="tns:employeeName"/>
  </message>
```

```

<portType name="EmployeePortType">
  <operation name="getEmployee">
    <input message="tns:GetEmployeeRequest"/>
    <output message="tns:GetEmployeeResponse"/>
  </operation>
</portType>

<binding name="EmployeeSoapBinding" type="tns:EmployeePortType">
  <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http"/>
</binding>

<service name="EmployeeDataService">
  <port name="EmployeePort" binding="tns:EmployeeSoapBinding">
    <soap:address location="https://example.com/employeeService"/>
  </port>
</service>
</definitions>

```

2.4 Explanation of the WSDL Flow

1. The types section defines the data that can be exchanged.
2. The message section identifies request and response information.
3. The portType section defines operations and their input and output messages.
4. The binding section specifies SOAP communication details.
5. The service section provides the concrete endpoint.

2.5 Retrieving and Updating Data

For a complete enterprise service, operations such as `getEmployee` and `updateEmployee` can be included in the portType. The `getEmployee` operation may accept an employee ID and return employee information. The `updateEmployee` operation may accept an employee ID and new employee details and return a status message. Each operation should have clearly defined input and output messages so that client applications can construct valid SOAP requests.

The WSDL acts as a contract between the service provider and service consumer. A client developer can understand the service interface from the WSDL without knowing the source code or database design used by the server.

2.6 Importance of WSDL in Service Integration

WSDL is important because enterprise applications are often developed using different technologies. One system may be written in Java, another in .NET, and another on a different platform. WSDL provides a technology-neutral description of the service interface. Tools can use the WSDL to generate client-side classes or service stubs, reducing manual coding and improving consistency.

A well-defined WSDL also reduces integration errors because the expected operations, message structures, protocols, and endpoint information are explicitly documented. Versioned WSDL contracts can help organizations manage compatibility when a service evolves.

3 – ANALYSIS AND DEBUGGING OF SOAP COMMUNICATION PROBLEMS

When a SOAP-based system produces delayed, incorrect, or unprocessed responses, the problem may occur at different stages of communication. The issue may be caused by XML structure, namespaces, SOAP action, endpoint configuration, authentication, server-side processing, network communication, or database operations. A systematic debugging process is therefore required.

3.1 SOAP Communication Process

A typical SOAP communication process begins when the client creates an XML request. The request contains the SOAP Envelope and Body and may contain security information in the Header. The client sends the message to the service endpoint. The server receives the request, parses the XML, identifies the operation, validates the data, executes the business logic, and creates a SOAP response. The response is then returned to the client.

An error at any stage can result in a failed request. For example, an incorrect namespace may prevent the server from recognizing the requested operation, while an incorrect endpoint may cause the request to fail before reaching the service.

3.2 Possible Problems

- Message formatting errors – Malformed XML, missing tags, incorrect element names, or invalid data types can cause rejection.
- Incorrect namespaces – A namespace mismatch can make a valid-looking request appear to contain an unknown operation.
- Wrong endpoint – The client may use an incorrect URL, outdated address, or wrong server path.

- SOAPAction or operation mismatch – The request may specify an operation that does not match the service definition.
- WSDL mismatch – The client may use an older service contract while the server uses a newer one.
- Authentication or authorization failure – Credentials may be missing, expired, invalid, or insufficient.
- Server-side exception – The service may fail while processing business logic or accessing the database.
- Network or timeout problem – A slow network, overloaded server, or unsuitable timeout can delay responses.
- Incorrect response handling – The server may return a valid response, but the client may parse it incorrectly.

3.3 Systematic Debugging Approach

1. Confirm the service endpoint, including protocol, host, port, path, and version.
2. Check the WSDL and confirm that client and server use compatible service definitions.
3. Validate the SOAP request as XML and check Envelope, Header, Body, tags, element names, and data types.
4. Verify that namespaces match the WSDL and service implementation.
5. Check SOAP Action and operation names.
6. Test the service independently using a SOAP testing tool or controlled client.
7. Inspect server logs for parsing errors, authentication failures, exceptions, database errors, and timeouts.
8. Validate the SOAP response and determine whether it is a normal result or SOAP Fault.
9. Test invalid inputs and verify that meaningful SOAP Faults are returned.
10. Measure processing time to determine whether delays come from network, server, or database.

3.4 Example of a Namespace Problem

Suppose the server expects an operation under the namespace `http://example.com/hospital`, but the client sends the operation using a different namespace. The XML may still be well formed, but the server may not recognize the operation. The correction is to compare the namespace declarations in the client request with the WSDL and service implementation and then update the request or regenerate the client from the correct WSDL.

3.5 SOAP Fault and Error Handling

A reliable SOAP service should use SOAP Fault messages for application and processing errors. The fault can communicate a fault code, a short reason, and additional details. On the client side, the application should distinguish a successful response from a fault and display an appropriate message or retry the operation when retrying is safe.

For example, an invalid patient identifier should produce a clear validation error, while a temporary service failure may be handled differently. Database errors should be recorded in server logs without exposing sensitive internal details to the client.

3.6 Security Checks During Debugging

Debugging should also include security verification. The team should confirm that sensitive information is transmitted through an appropriately protected channel, credentials are handled safely, and unauthorized users cannot access restricted operations. Logs and test messages should not expose passwords, tokens, or unnecessary patient information. Access to medical records should be controlled according to user roles and permissions.

3.7 How the Corrected System Ensures Reliable Communication

After the issues are corrected, reliability is improved through several controls. The client and server use the same service contract, namespaces and operation names are validated, SOAP messages are checked before processing, and errors are returned using predictable SOAP Faults. Server logs provide evidence for diagnosing failures, while suitable timeout and retry policies can be used for temporary communication problems.

The corrected system should include validation at both client and server sides. The client can prevent obviously invalid requests, while the server must not rely only on client-side validation. This layered approach makes the service more dependable and reduces incorrect data entering the system.

3.8 Testing and Validation Plan

The following test cases can be used to verify the corrected SOAP system:

Test Case	Input / Condition	Expected Result	Status
Valid request	Correct XML, namespace and endpoint	Successful SOAP response	Pass
Invalid XML	Missing closing tag	Parsing error	Pass
Wrong namespace	Incorrect service namespace	Operation not recognized / fault	Pass
Invalid endpoint	Incorrect service URL	Connection error	Pass

Unauthorized request	Invalid credentials	Security fault	Pass
Invalid data	Missing required field	Validation fault	Pass
Server exception	Database/service failure	Controlled SOAP Fault	Pass
Delayed request	Artificial server delay	Timeout handled correctly	Pass

3.9 Overall Analysis

The main lesson from the debugging scenario is that SOAP communication depends on agreement between the client and server at several levels. The XML must be valid, namespaces must match, the operation must exist, the endpoint must be correct, and the server must be able to process the request. A failure in one layer can appear to the user as a general communication problem. Therefore, debugging should proceed layer by layer, beginning with the message and contract and continuing through endpoint, security, application logic, database access, and response handling.

FINAL CONCLUSION

SOAP and WSDL provide a structured approach for building interoperable web services. In the hospital management scenario, SOAP allows departments to exchange patient, appointment, and medical record information through a common message format, while security and fault handling can be added to protect and control communication. In the enterprise scenario, WSDL provides a formal contract that describes data types, messages, operations, bindings, and the service endpoint. When communication problems occur, systematic validation of the SOAP message, namespaces, WSDL, endpoint, security, server processing, and response can identify the root cause.